

Docker Compose YAML Structure Guide

Tags, Sub-tags, and How Deep They Go

Docker Compose YAML Structure Guide

This guide breaks down the structural skeleton of a `docker-compose.yml` file — no values, just the shape: what tags exist, what sub-tags live under them, and how deep the nesting goes.

Top-Level Tags

```
version          (optional in modern Compose, often omitted now)
services
networks
volumes
configs
secrets
```

Unlike Kubernetes (fixed 4 tags), Compose only *requires* `services` — the rest are optional and only appear if you use them.

1. services

This is the big one — each service is a name, and under it:

```
services
├── <service_name>
│   ├── image
│   ├── build
│   │   ├── context
│   │   ├── dockerfile
│   │   ├── args
│   │   │   └── (key: value pairs)
│   │   ├── target
│   │   └── cache_from
│   ├── container_name
│   ├── command
│   ├── entrypoint
│   ├── environment
│   │   └── (key: value pairs, or list of KEY=VALUE)
│   ├── env_file
│   ├── ports
│   │   └── (list of "HOST:CONTAINER" or objects: target/published/protocol)
│   ├── expose
│   ├── volumes
│   │   └── (list of "HOST:CONTAINER" or long-form objects: type/source/target)
│   ├── networks
│   │   └── <network_name>
│   │       ├── aliases
│   │       └── ipv4_address
│   ├── depends_on
│   │   └── <service_name>
│   │       └── condition
│   ├── restart
│   ├── deploy
│   │   ├── replicas
│   │   ├── resources
│   │   │   ├── limits
│   │   │   │   ├── cpus
│   │   │   │   └── memory
│   │   │   └── reservations
│   │   │       ├── cpus
│   │   │       └── memory
│   │   ├── restart_policy
│   │   │   ├── condition
│   │   │   ├── delay
│   │   │   └── max_attempts
│   │   └── placement
│   │       └── constraints
```

```

├── healthcheck
│   ├── test
│   ├── interval
│   ├── timeout
│   ├── retries
│   └── start_period
├── logging
│   ├── driver
│   └── options
├── labels
├── links
├── user
├── working_dir
└── (more advanced/rare: cap_add, cap_drop, devices, secrets, configs, network_mode, extra_hosts, ulimits,
    security_opt)

```

2. networks

```

networks
├── <network_name>
│   ├── driver
│   ├── driver_opts
│   ├── external
│   ├── ipam
│   │   ├── driver
│   │   │   ├── config
│   │   │   │   ├── subnet
│   │   │   │   └── gateway
│   └── labels

```

3. volumes

```

volumes
├── <volume_name>
│   ├── driver
│   ├── driver_opts
│   ├── external
│   └── labels

```

4. configs / secrets

Same shape for both:

```

configs / secrets
├── <name>
│   ├── file
│   └── external

```

The Pattern to Remember

- **Top-level = up to 5 tags**, but only services is required.
- **services.<name>** is where almost all the complexity lives — it's the Compose equivalent of Kubernetes' spec.

Getting the Exact, Real Schema

There's no `kubectrl explain` equivalent for Compose, but there are two reliable ways to check the real, current schema instead of memorizing:

```
docker compose config
```

This validates your file and prints the fully resolved configuration.

Alternatively, check the [Compose Spec](#) on GitHub for the authoritative, up-to-date field list.